AgroRiego — Entrega del Sprint 1

Nov 8, 2026 · @Julian

Métodos Ágiles de Desarrollo · Dr. Carlos Jesús Hinojosa Rodríguez · Equipo 3: Alejandro Erro González, Christian Daniel Valencia Patrón, Julian Espinosa Llanos, Ricardo Orduño Camacho y Paul Candelario Seañez Villalobos · Sprint 1: 26 de octubre al 6 de noviembre de 2026

1. Historias entregadas

El Sprint 1 se comprometió a 3 historias (11 SP). Las 3 están construidas y sus 10 criterios de aceptación pasan en pruebas automatizadas. Las historias completas con sus criterios están en docs/HISTORIAS.md.

Historia	SP	Criterios de aceptación	Pantallas	Estado en la entrega
HU-01 · Iniciar sesión. Como productor, quiero iniciar sesión con mi correo y contraseña, para acceder solo yo a mis parcelas.	3	4: acceso, mensaje genérico, bloqueo tras 5 intentos, restablecimiento	Inicio de sesión, restablecer contraseña	Construida y probada
HU-03 · Registrar parcela. Como productor, quiero registrar mis parcelas con nombre, superficie, cultivo y ubicación, para organizar el monitoreo por terreno.	3	3: alta, campos obligatorios, edición	Panel de parcelas, nueva y editar parcela con mapa	Construida y probada

Historia	SP	Criterios de aceptación	Pantallas	Estado en la entrega
HU-04 · Vincular sensor y válvula. Como técnico de campo, quiero vincular un sensor y una válvula a una parcela mediante su código, para que la parcela envíe lecturas y se pueda regar.	5	3: vincular por código o QR, rechazo de código ya vinculado, «En línea» con la primera lectura	Equipos del técnico con escaneo QR	Construida y probada; falta probar un QR físico (issue #3)

Rebanadas fuera del sprint, como estaba planeado: humedad en tiempo real, riego y alertas (Sprint 2), y riego programado y automático (Sprint 3).

2. Diseño y arquitectura de la solución

La solución es un **monolito modular**: una app web instalable en el celular (PWA), una API en Node.js con módulos separados y una capa IoT por MQTT, desplegados con Docker Compose en una instancia EC2 de AWS. Los diagramas de componentes y de despliegue, con la función de cada componente y nodo, están en el **Anexo A** (PDF).

Lo que se construyó en este sprint, por componente:

Componente	Qué se construyó	Dónde
App AgroRiego (PWA)	Inicio de sesión, restablecer contraseña, panel y registro de parcelas con mapa, pantalla del técnico con escaneo QR	`web/src/`
Módulo de autenticación	Sesión con JWT en cookie httpOnly, bcrypt, bloqueo de 15 min tras 5 intentos, enlaces de restablecimiento de 1 h	`api/src/modulos/autenticacion.js`

Componente	Qué se construyó	Dónde
Módulo de parcelas	Alta, edición y validación; cada productor ve solo sus parcelas	`api/src/modulos/parcelas.js`
Módulo de dispositivos	Vinculación por código, rechazo de duplicados, estados del equipo	`api/src/modulos/dispositivos.js`
Adaptador IoT	Cliente MQTT que recibe lecturas y pone el sensor «En línea»	`api/src/iot/adaptador.js`
Acceso a datos	PostgreSQL 16 con 3 migraciones SQL versionadas	`api/migrations/`
Sensores (simulados)	Simulador que publica lecturas cada 5 min o cada 10 s	`simulador/`
Despliegue	Docker Compose (nginx, API, PostgreSQL, Mosquitto) e instalación automática en EC2	`docker-compose.yml`, `deploy/`

Decisión de arquitectura del sprint (ADR-006): JavaScript y SQL con node-postgres en lugar de TypeScript y Prisma, para tener menos pasos de compilación y migraciones legibles en cada revisión.

3. Casos de prueba

Se diseñaron 33 casos que cubren los 10 criterios de aceptación; los 33 pasaron. El diseño completo (estrategia, datos de prueba, pasos y resultado esperado) está en el **Anexo B** (PDF).

Historia	Casos	Pruebas automatizadas	Resultado	Detalle por caso
HU-01 Iniciar sesión	13	23 (API 13 · interfaz 10)	13 de 13 pasan	HU-01.md
HU-03 Registrar parcela	11	18 (API 11 · interfaz 7)	11 de 11 pasan	HU-03.md

Historia	Casos	Pruebas automatizadas	Resultado	Detalle por caso
HU-04 Vincular sensor y válvula	9	16 (API 9 · MQTT 2 · interfaz 5)	9 de 9 pasan	HU-04.md
Total	**33**	**57**	**33 de 33**	

Niveles: pruebas de la API contra PostgreSQL 16 real (Vitest + Supertest), integración IoT con un broker MQTT en memoria y pruebas de interfaz con Testing Library. Los casos funcionales se repiten a mano sobre la app desplegada en la demostración (sección 7).

4. Evidencia de ejecución en la herramienta de seguimiento a errores

Las pruebas se ejecutan en **GitHub Actions** en cada push y en cada pull request, y los defectos se registran en **GitHub Issues**. En la primera corrida se encontró 1 defecto; se corrigió y la segunda corrida pasó completa.

Fecha	Evidencia	Resultado
1 nov	Corrida #41402139029	API 35/35 · interfaz 21/22 · 1 falla en CP-03-02
1 nov	Issue #2 · bug: CP-03-02, la prueba buscaba un campo antes de que cargara el formulario	Causa en la prueba, no en la app; corregido el 1 nov en el commit 212aa7f. El issue se registró y cerró el 8 nov
1 nov	Corrida #41402269550	API 35/35 · interfaz 22/22 · simulador 3/3 · imágenes Docker OK
8 nov	PR #1: las 4 verificaciones corren de nuevo al abrir el pull request (corrida #42547812321)	En verde
8 nov	Issue #3 · enhancement: falta probar un QR físico con la cámara (CP-04-02)	Abierto; pasa al Sprint 2 (no se probó con un QR físico en la demostración)

5. Código de la aplicación en el repositorio

Repositorio público: **github.com/Junsa64/AgroRiego**. El código del sprint está en la rama `sprint-1`, con un commit por tarea del Kanban (T1.2 a T1.14), y se integra a `main` con el pull request #1.

Carpeta	Contenido
`api/`	API Express + PostgreSQL: módulos de autenticación, parcelas y dispositivos, adaptador MQTT, migraciones y 35 pruebas
`web/`	App móvil (PWA) en React: 5 pantallas y 22 pruebas
`simulador/`	Sensores simulados que publican lecturas por MQTT, con 3 pruebas
`deploy/`	Guía y script de instalación en AWS EC2
`.github/workflows/ci.yml`	Integración continua: pruebas de API, interfaz, simulador e imágenes Docker
`docs/`	Historias, prototipo, arquitectura, diseño y resultados de pruebas, tableros de seguimiento

Para correrlo: `cp .env.example .env && docker compose up -d --build`, luego `docker compose exec api npm run seed`. Usuarios de prueba: `productor@agroriego.test` y `tecnico@agroriego.test`, contraseña `Prueba#2026`.

El código se generó con apoyo de IA (Claude Code) a partir de las historias, el prototipo y la arquitectura del equipo; los integrantes lo revisaron por área (ver BiWeekly 2).

6. Tablero Kanban, Niko-Niko y Burndown

Los tres tableros completos están en el **Anexo C** y en docs/seguimiento/ del repositorio.

Herramienta	Estado al cierre del Sprint 1
Tablero Kanban	Sprint 0: 6 de 6 tareas en «Hecho». Sprint 1: T1.1 en «Hecho», T1.3 a T1.14 en «En pruebas» y T1.2 en «En progreso».

Herramienta	Estado al cierre del Sprint 1
Niko-Niko	10 días registrados de los 5 integrantes: Julian, Erro y Paul «Feliz»; Daniel y Ricardo «Neutral». Nota: Daniel, por carga de otras materias.
Burndown chart	11 SP restantes los 10 días: ninguna historia llegó a «Hecho» antes del 6 de noviembre.

7. Demostración del producto

Ambiente. Una instancia EC2 **t3.micro** (elegible del nivel gratuito) en AWS us-west-2, con Docker Compose: nginx, API, PostgreSQL y Mosquitto. Solo el puerto 80 está abierto; la base de datos y MQTT no se exponen. Se desplegó con `deploy/instalar-ec2.sh` desde la rama `sprint-1` (commit `f350aea`, la misma del PR #1) y está programada para apagarse sola el 9 de noviembre a las 23:59 (hora de Sonora) para no consumir créditos. Al volver a encenderla cambia la IP pública.

- URL usada en la demostración: http://34.219.182.88
- Productor: `productor@agroriego.test` · Técnico: `tecnico@agroriego.test` · contraseña `Prueba#2026` (datos de prueba del seed)

Guion ejecutado el 8 de noviembre de 2026 sobre la instancia desplegada, en vista de celular (375 px):

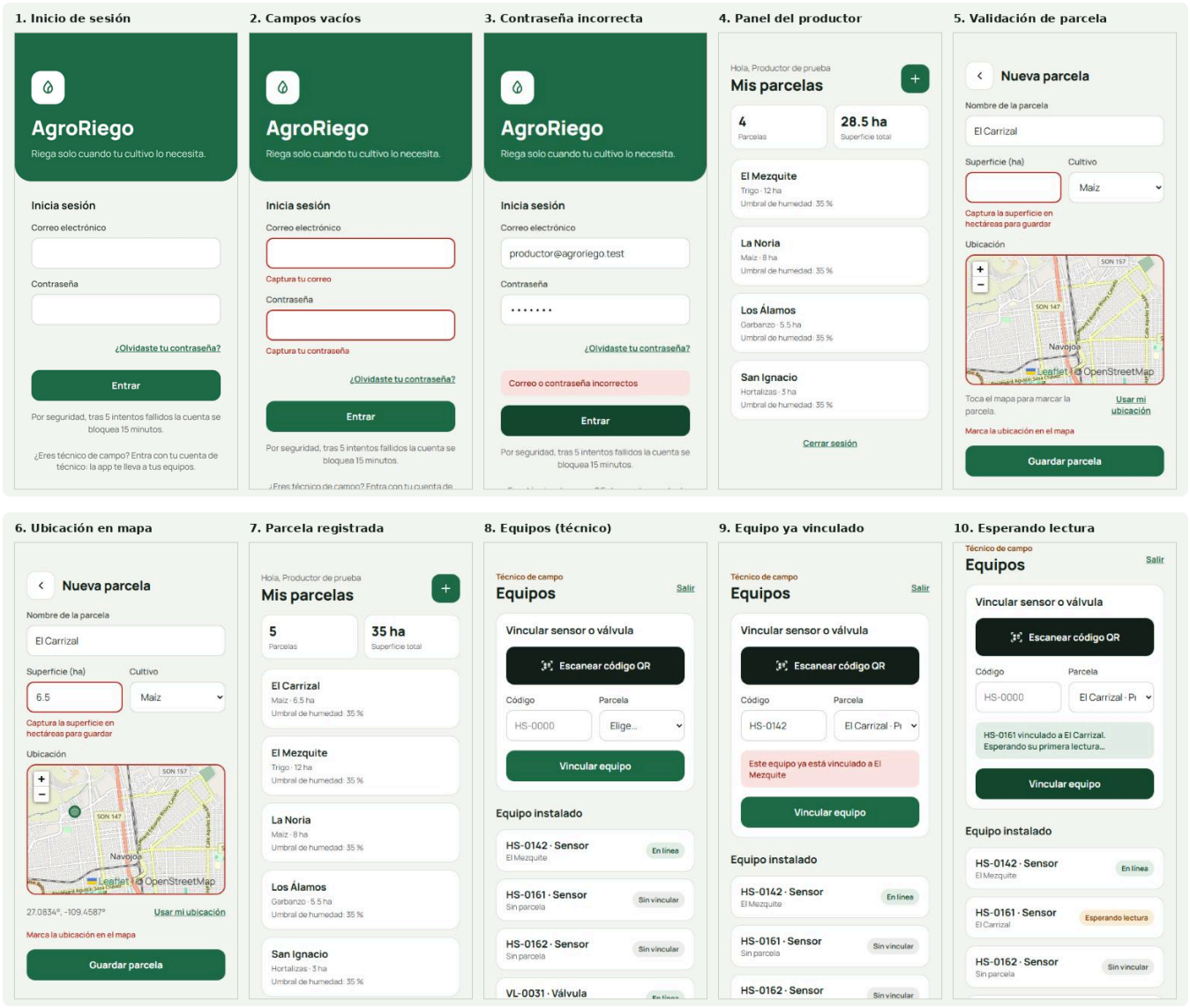
#	Historia	Qué se hizo	Resultado observado	Caso
1	HU-01	Abrir la app sin sesión	Pantalla «Inicia sesión»	CP-01-12
2	HU-01	Pulsar «Entrar» sin capturar datos	«Captura tu correo» y «Captura tu contraseña»	CP-01-05
3	HU-01	Productor con contraseña incorrecta	«Correo o contraseña incorrectos», no entra	CP-01-03
4	HU-01	Productor con datos correctos	Panel «Mis parcelas» con 4 parcelas	CP-01-01
5	HU-03	Guardar parcela nueva sin superficie ni ubicación	No guarda; pide la superficie y marcar el mapa	CP-03-03, CP-03-04

#	Historia	Qué se hizo	Resultado observado	Caso
6	HU-03	Capturar «El Carrizal», 6.5 ha, maíz, y marcar el punto en el mapa	Se guarda y aparece en el panel: 5 parcelas, 35 ha	CP-03-01
7	HU-01 / HU-04	Técnico inicia sesión	Pantalla «Equipos» con el equipo instalado y su estado en texto	CP-01-02
8	HU-04	Vincular HS-0142 a El Carrizal	Rechazado: «Este equipo ya está vinculado a El Mezquite»	CP-04-04
9	HU-04	Vincular HS-0161 a El Carrizal	«HS-0161 vinculado a El Carrizal. Esperando su primera lectura…»; en la lista aparece «Esperando lectura»	CP-04-01

Lo que no se mostró en vivo y cómo se verificó. El paso a «En línea» de HS-0161 necesita correr el simulador dentro de la instancia, y para eso hay que abrir SSH (puerto 22), que se dejó cerrado. Ese comportamiento lo cubren las pruebas automáticas CP-04-07 (incluida la de MQTT, menos de 10 s) que pasan en GitHub Actions. Para la demostración en clase: `ssh ubuntu@<IP>` y `npm run simular -- --codigo HS-0161 --rapido` según `deploy/README.md`, sección 3. El escaneo con un QR físico sigue pendiente (issue #3).

Observación de la demostración. En el paso 6, los mensajes de error del intento anterior siguen visibles hasta que se vuelve a pulsar «Guardar parcela»; al guardar desaparecen. Se propone limpiarlos en cuanto el campo se corrige (mejora menor para el Sprint 2).

Capturas de la demostración:



8. Resultado del sprint y aprendizajes

Velocidad del Sprint 1: 0 de 11 SP dentro del sprint. El código y las pruebas de las 3 historias quedaron listos el 1 de noviembre, pero al 6 de noviembre ninguna cumplía la Definición de Terminado: faltaban la aprobación del pull request, el despliegue y la aceptación del Product Owner. El despliegue se completó el 8 de noviembre (sección 7); la aprobación del pull request por otro integrante y la aceptación del Product Owner quedan para la revisión del sprint, y solo entonces las historias pasan a «Hecho».

Puntos para la retrospectiva (por confirmar con el equipo):

- **El cuello de botella fue la revisión, no la programación.** Propuesta: revisar e integrar cada pull request en máximo 2 días hábiles.

- **Un solo integrante concentró los commits.** Propuesta: en el Sprint 2 cada responsable abre el pull request de sus propias tareas.
- **El código se generó con IA.** Propuesta: cada integrante explica su módulo en la junta antes de aprobarlo, para que el equipo domine lo que entrega.
- **El despliegue se dejó para el final.** Propuesta: desplegar en AWS desde el primer día del Sprint 2 y actualizar con cada integración.

Anexo A

Diseño y arquitectura de la solución

Diagramas de componentes y de despliegue (docs/AgroRiego_Arquitectura_Preliminar.pdf)

AgroRiego — Arquitectura preliminar

Nov 1, 2026 · @Julian

Métodos Ágiles de Desarrollo · Dr. Carlos Jesús Hinojosa Rodríguez · Equipo 3

AgroRiego se plantea como un **monolito modular**: una sola API con módulos bien separados (autenticación, parcelas, dispositivos, riego y alertas), una app web instalable para el celular y una capa IoT por MQTT. Esta arquitectura es preliminar: se decide lo mínimo para el Sprint 1 y se revisa al cierre de cada sprint.

Principios de arquitectura continua

La arquitectura continua trata la arquitectura como algo que evoluciona con cada entrega, guiado por requisitos de calidad medibles y decisiones registradas. Así se aplican sus seis principios en el proyecto:

Principio	Cómo se aplica en AgroRiego
1. Arquitecturar productos, no solo proyectos	La arquitectura sigue al producto completo (las 16 historias), aunque en estos 3 sprints solo se construyen 9. Los módulos de consumo y reportes ya tienen lugar previsto.
2. Enfocarse en los atributos de calidad, no solo en funciones	Se definen requisitos medibles: la válvula responde en menos de 30 s, el panel se actualiza sin recargar, la app se usa en exteriores y con mala señal. Ver la última sección.
3. Retrasar las decisiones hasta que sean necesarias	La nube ya está elegida (AWS), pero se empieza con lo mínimo: una sola instancia EC2 con contenedores. Los servicios gestionados (Amazon RDS, AWS IoT Core), la separación en microservicios y el hardware real se deciden después; mientras, se usa un simulador de sensores.

Principio	Cómo se aplica en AgroRiego
4. Arquitecturar para el cambio: aprovechar el poder de lo pequeño	Módulos con fronteras claras y comunicación por eventos MQTT, para que cualquiera pueda separarse en un servicio propio sin reescribir los demás.
5. Arquitecturar para construir, probar, desplegar y operar	Integración continua en GitHub Actions, pruebas automatizadas por criterio de aceptación, despliegue en contenedores y monitoreo de los equipos en campo desde el Sprint 3.
6. Modelar la organización del equipo según el diseño del sistema	Cada módulo tiene un responsable natural: base de datos (Alejandro Erro), API (Daniel Valencia), nube e IoT (Julian Espinosa), app (Paul Seañez).

Las decisiones se registran como ADR (registros de decisión de arquitectura) en `docs/adr/` del repositorio, y se revisan en cada retrospectiva.

Diagrama de componentes

La API modular concentra la lógica; el campo solo habla con ella por MQTT

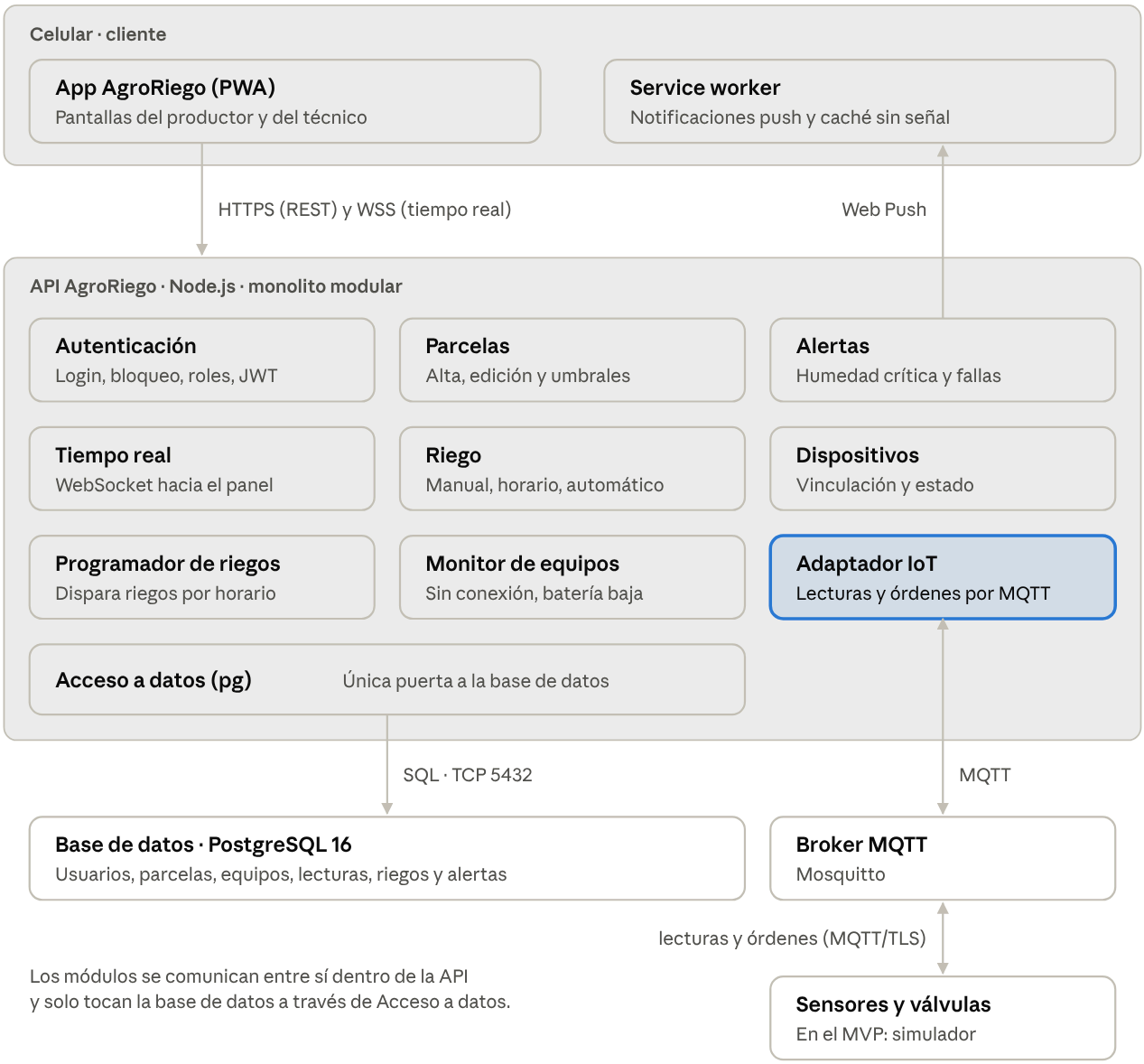


Diagrama de componentes · cliente, API con 10 módulos, datos e IoT

La app solo habla con la API. Los equipos de campo solo hablan con el broker MQTT, y el Adaptador IoT (resaltado) es el único módulo que traduce entre ese mundo y el resto del sistema.

Función de cada componente

Componente	Función	Se comunica con	Historias	Sprint
App AgroRiego (PWA)	Interfaz móvil del productor y del técnico: inicio de sesión, panel de parcelas, riego, alertas y alta de equipos. Se instala en el celular desde el navegador.	Módulos de la API por HTTPS (REST); servicio de tiempo real por WebSocket	Todas	1 a 3
Service worker	Parte de la PWA que recibe las notificaciones push aunque la app esté cerrada y guarda en caché la interfaz para abrirla con mala señal.	Módulo de alertas (Web Push)	HU-11, HU-12	2
Módulo de autenticación	Valida correo y contraseña, cuenta intentos fallidos, bloquea la cuenta 15 min, emite la sesión (JWT en cookie) y gestiona el restablecimiento de contraseña. Protege las rutas de los demás módulos según el rol.	Acceso a datos; servicio de correo	HU-01	1
Módulo de parcelas	Alta, consulta y edición de parcelas con sus umbrales de humedad. Garantiza que cada productor vea solo las suyas.	Acceso a datos	HU-03	1

Componente	Función	Se comunica con	Historias	Sprint
Módulo de dispositivos	Vincula sensores y válvulas a una parcela por código o QR, rechaza códigos duplicados y lleva el estado de cada equipo (esperando lectura, en línea, sin conexión).	Adaptador IoT; acceso a datos	HU-04	1
Adaptador IoT	Cliente MQTT de la API: recibe las lecturas de humedad, temperatura y batería, y publica las órdenes de abrir o cerrar válvulas. Aísla al resto del sistema del protocolo del hardware.	Broker MQTT; módulos de dispositivos, riego y alertas	HU-04, HU-05, HU-07	1 y 2
Servicio de tiempo real	Envía a la app cada lectura nueva y cada cambio de estado de riego, para que el panel se actualice sin recargar.	App (WebSocket); adaptador IoT	HU-05, HU-07	2
Módulo de riego	Abre y cierra la válvula a petición del productor, por horario o automáticamente por umbral de humedad, y registra cada riego con su duración.	Adaptador IoT; programador; acceso a datos	HU-07, HU-08, HU-09	2 y 3
Programador de riegos	Revisa cada minuto los programas por horario y dispara el inicio y fin de cada riego programado.	Módulo de riego	HU-08	3

Componente	Función	Se comunica con	Historias	Sprint
Módulo de alertas	Detecta humedad crítica, evita repetir la misma alerta en 1 h y envía las notificaciones push.	Adaptador IoT; monitor de equipos; service worker	HU-11, HU-12	2 y 3
Monitor de equipos	Marca «Sin conexión» a los equipos que llevan 30 min sin reportar y genera avisos de batería baja para el técnico.	Módulo de dispositivos; módulo de alertas	HU-12	3
Acceso a datos (node-postgres)	Conexión única a la base de datos (pool de node-postgres) y migraciones SQL versionadas; cada módulo tiene sus consultas en su propio archivo y ninguno toca las tablas de otro. Facilita cambiar o separar la base de datos más adelante.	Base de datos	Todas	1
Base de datos (PostgreSQL)	Guarda usuarios, parcelas, dispositivos, lecturas, riegos, programas y alertas.	Acceso a datos	Todas	1
Broker MQTT (Mosquitto)	Intermediario de mensajes entre los equipos de campo y la API. Los equipos publican lecturas y se suscriben a sus órdenes.	Adaptador IoT; sensores y válvulas	HU-04, HU-05, HU-07	1

Componente	Función	Se comunica con	Historias	Sprint
Sensores y válvulas	En campo: el sensor mide humedad y temperatura del suelo cada 5 min; la válvula abre o cierra el agua. En el MVP se reemplazan por el **simulador**, que publica los mismos mensajes.	Broker MQTT	HU-04, HU-05, HU-07	1

Servicios externos: un proveedor de correo para los enlaces de restablecimiento (en el MVP se muestran en la consola) y el servicio de notificaciones push del navegador.

Diagrama de despliegue

Todo el backend corre en contenedores en la nube; los datos quedan en red privada

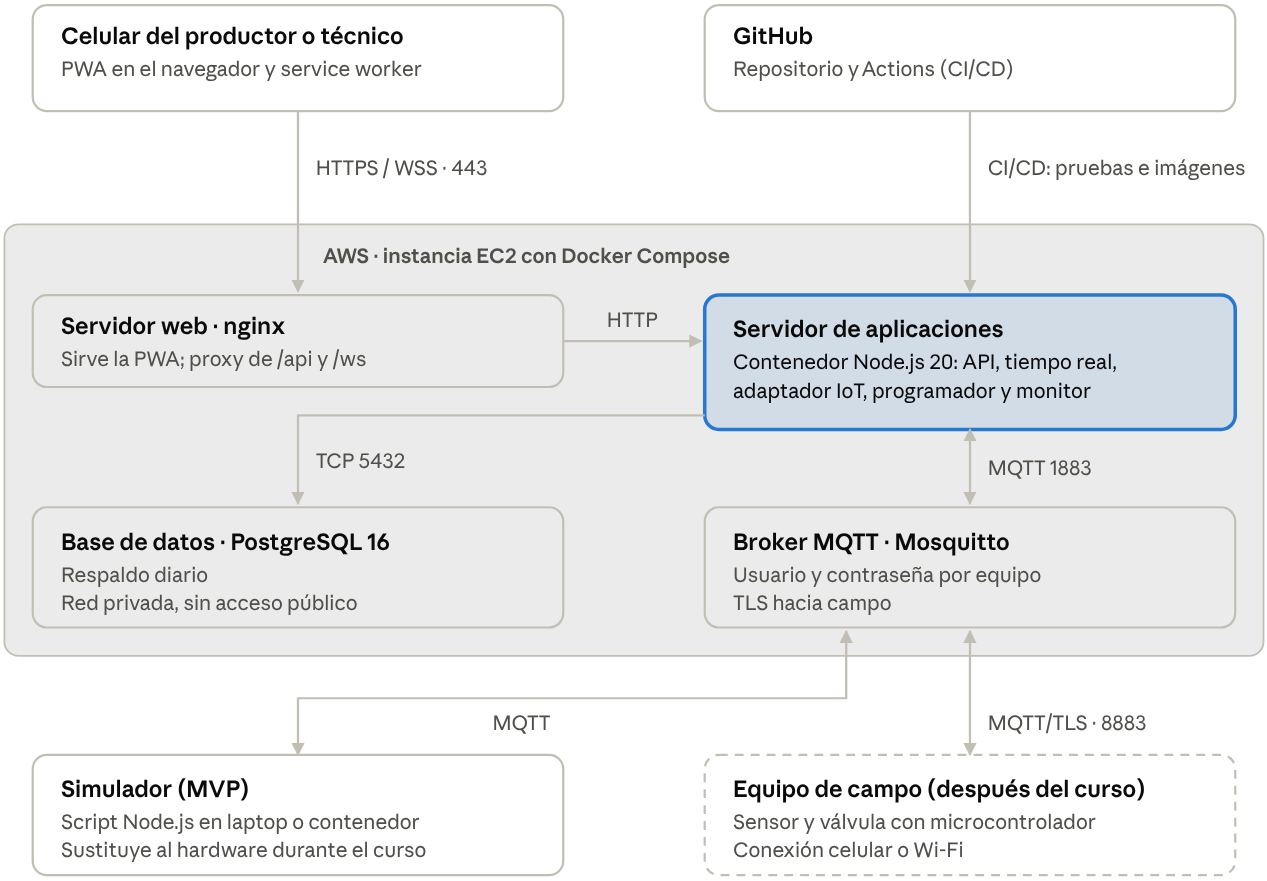


Diagrama de despliegue · 8 nodos en celular, nube, GitHub y campo

El celular solo entra por el servidor web en el puerto 443. La base de datos y el broker quedan en la red privada de la nube; desde campo solo se expone el puerto MQTT con TLS.

Función de cada nodo

Nodo	Qué se despliega ahí	Función	Comunicación
Celular del productor o técnico	Navegador móvil con la PWA instalada y su service worker	Ejecuta la interfaz y recibe las notificaciones push	HTTPS y WSS (puerto 443) hacia la nube

Nodo	Qué se despliega ahí	Función	Comunicación
Servidor web	Contenedor nginx con los archivos estáticos de la PWA	Entrega la app al celular y redirige `/api` y `/ws` al servidor de aplicaciones	HTTPS 443 desde internet; HTTP interno hacia la API
Servidor de aplicaciones	Contenedor Docker con Node.js 20: la API con todos sus módulos, el servicio de tiempo real, el adaptador IoT, el programador y el monitor	Ejecuta toda la lógica de negocio	HTTP/WS interno; TCP 5432 a la base de datos; MQTT 1883 al broker dentro de la red privada
Servidor de base de datos	PostgreSQL 16 con respaldo diario	Almacena todos los datos; no es accesible desde internet	TCP 5432, solo desde el servidor de aplicaciones
Broker MQTT	Contenedor Mosquitto con usuario y contraseña por equipo	Recibe lecturas y distribuye órdenes de riego	MQTT sobre TLS 8883 desde campo; 1883 interno
Equipo de campo	Sensor de humedad y válvula con un microcontrolador con conexión celular o Wi-Fi	Mide la humedad cada 5 min y abre o cierra el agua	MQTT/TLS 8883 hacia el broker
Simulador (MVP)	Script Node.js en una laptop o contenedor	Sustituye al equipo de campo mientras no hay hardware; publica lecturas y responde a las órdenes	MQTT hacia el broker
GitHub (repositorio y Actions)	Código fuente y pipeline de integración continua	En cada pull request corre las pruebas; al integrar a `main` construye las imágenes y despliega al ambiente de pruebas	HTTPS hacia la nube

En el Sprint 1 todos los nodos de nube corren con Docker Compose en una sola instancia Amazon EC2, pagada con los créditos de AWS del equipo y con una alerta de AWS Budgets para no rebasarlos. Como cada uno es un contenedor separado, después pueden moverse a servicios gestionados (Amazon RDS para PostgreSQL y AWS IoT Core para MQTT) sin cambiar el código.

Decisiones de arquitectura y requisitos de calidad

Requisitos de calidad

Atributo	Escenario medible	Cómo lo atiende la arquitectura
Rendimiento	La válvula se abre en menos de 30 s después de confirmar el riego	Órdenes por MQTT y confirmación de estado desde el equipo
Oportunidad de datos	Una lectura nueva aparece en el panel en menos de 10 s sin recargar	Servicio de tiempo real por WebSocket
Disponibilidad	Si un equipo deja de reportar 30 min, el técnico recibe aviso	Monitor de equipos y alertas push
Seguridad	Solo el dueño ve y controla sus parcelas; cuenta bloqueada tras 5 intentos	Módulo de autenticación con roles; base de datos sin acceso público; MQTT con TLS
Usabilidad en campo	Se usa en exteriores con un celular y señal irregular	PWA con caché, alto contraste y botones de 44 px o más
Modificabilidad	Un módulo puede separarse en servicio propio sin reescribir los demás	Monolito modular, acceso a datos centralizado y eventos MQTT

Decisiones registradas (ADR)

ADR	Decisión	Motivo	Se revisa en
ADR-001	Monolito modular en lugar de microservicios	Equipo de 5 personas y 6 semanas; menos complejidad de despliegue	Retrospectiva del Sprint 3
ADR-002	MQTT para comunicar los equipos de campo	Protocolo ligero, pensado para conexiones inestables y equipos de bajo consumo	Cuando se elija el hardware real
ADR-003	PWA en lugar de app nativa	Una sola base de código para Android y iPhone, sin tiendas de apps	Al terminar el Sprint 2
ADR-004	AWS: una instancia EC2 con Docker Compose	Cabe en los créditos de AWS del equipo; por usar contenedores, se puede pasar a RDS o IoT Core sin cambiar el código	Si el uso se acerca al límite de créditos
ADR-005	Simulador en lugar de hardware en el MVP	No se cuenta con sensores ni válvulas durante el curso	Después del curso

Cambio registrado durante el Sprint 1 (arquitectura continua: la decisión se ajustó al construir):

ADR	Decisión	Motivo	Se revisa en
ADR-006	JavaScript (módulos ES) y SQL con node-postgres en lugar de TypeScript y Prisma	Menos pasos de compilación y herramientas para un equipo nuevo en el stack; las migraciones quedan como SQL legible y revisable en cada pull request	Retrospectiva del Sprint 2

Anexo B

Diseño de casos de prueba del Sprint 1
(docs/pruebas/AgroRiego_Diseno_Pruebas_Sprint1.pdf)

AgroRiego — Diseño de pruebas del Sprint 1

Nov 1, 2026 · @Julian

Métodos Ágiles de Desarrollo · Dr. Carlos Jesús Hinojosa Rodríguez · Equipo 3

El Sprint 1 (26 oct – 6 nov 2026) se valida con 33 casos de prueba que cubren los 10 criterios de aceptación de HU-01, HU-03 y HU-04. Cada criterio tiene al menos un caso positivo y uno negativo o de límite.

Estrategia de pruebas

Alcance

Historia	Qué se prueba	Fuera de alcance en este sprint
HU-01 Iniciar sesión	Acceso, mensaje genérico, bloqueo tras 5 intentos, restablecimiento de contraseña, rutas protegidas	Gestión de usuarios y roles (HU-02)
HU-03 Registrar parcela	Alta, validaciones, edición y aislamiento entre productores	Lecturas de humedad en el panel (HU-05)
HU-04 Vincular sensor y válvula	Vinculación por código y QR, duplicados, permisos y cambio a «En línea»	Alertas de falla de equipo (HU-12)

Niveles de prueba

Nivel	Qué cubre	Herramienta	Responsable
Unitarias	Reglas de negocio: bloqueo, validaciones, estados del dispositivo	Vitest	Quien programa la tarea
Integración de API	Cada endpoint contra la base de datos de prueba	Vitest + Supertest	Daniel Valencia, Alejandro Erro

Nivel	Qué cubre	Herramienta	Responsable
Integración IoT	Lectura MQTT del simulador → cambio de estado del dispositivo	Vitest + broker Mosquitto de prueba	Julian Espinosa
Funcionales (manuales)	Flujo completo en la app, como lo usaría el productor o el técnico	Navegador móvil (390 px)	Paul Seañez, Alejandro Erro

Los defectos se registran en GitHub Issues como *Bug* ligado a su historia, con el ID del caso de prueba, pasos para reproducirlo y captura de pantalla.

Ambiente y datos de prueba

- **Ambiente:** API y app en el ambiente de pruebas en AWS (instancia EC2), con PostgreSQL y broker MQTT en Docker; sensores y válvulas simulados con el script del simulador.
- **Usuarios semilla:** productor `productor@agroriego.test` y técnico `tecnico@agroriego.test`, ambos con contraseña `Prueba#2026`; un segundo productor `productor2@agroriego.test` para probar aislamiento de datos.
- **Parcelas semilla:** El Mezquite, Los Álamos, La Noria y San Ignacio (del productor 1).
- **Dispositivos semilla:** sensores `HS-0161` y `HS-0162` sin vincular, sensor `HS-0142` ya vinculado a El Mezquite, válvula `VL-0040` sin vincular.
- La base de datos se reinicia con la semilla antes de cada corrida completa.

Criterios de entrada y salida

- **Entrada:** la tarea está en «En pruebas» en el Kanban, su pull request está abierto y el ambiente de pruebas tiene la versión de ese PR.
- **Salida:** todos los casos de prioridad Alta pasan, no quedan defectos críticos abiertos y las pruebas automatizadas pasan en GitHub Actions.

En las tablas, la columna Estado empieza en «Pendiente» y se actualiza al ejecutar cada caso.

HU-01 · Iniciar sesión

Como productor, quiero iniciar sesión con mi correo y contraseña, para acceder solo yo a la información y control de mis parcelas.

Criterios de aceptación: **CA1** acceso con datos correctos · **CA2** mensaje genérico con datos incorrectos · **CA3** bloqueo de 15 min tras 5 intentos fallidos · **CA4** restablecimiento de contraseña por correo.

ID	CA	Escenario	Pasos y datos	Resultado esperado	Prioridad	Estado
CP-01-01	CA1	Productor entra con datos correctos	Capturar `productor@agroriego.test` / `Prueba#2026` y pulsar Entrar	Abre el panel de parcelas con el nombre del productor	Alta	Pasó
CP-01-02	CA1	Técnico entra con datos correctos	Capturar `tecnico@agroriego.test` / `Prueba#2026` y pulsar Entrar	Abre la pantalla de Equipos del técnico	Alta	Pasó
CP-01-03	CA2	Contraseña incorrecta	Correo válido con contraseña `Mala123`	Mensaje «Correo o contraseña incorrectos»; no entra	Alta	Pasó
CP-01-04	CA2	Correo no registrado	Capturar `noexiste@agroriego.test` / `Prueba#2026`	Mismo mensaje genérico, sin revelar que el correo no existe	Alta	Pasó
CP-01-05	CA2	Campos vacíos	Pulsar Entrar sin capturar nada	Se marcan ambos campos como obligatorios; no se llama a la API	Media	Pasó
CP-01-06	CA3	Bloqueo tras 5 intentos fallidos	Fallar la contraseña 5 veces seguidas; en el 6.º intento usar la contraseña correcta	El 6.º intento se rechaza con «Cuenta bloqueada. Intenta en 15 minutos»	Alta	Pasó
CP-01-07	CA3	Desbloqueo al pasar 15 min	Con la cuenta bloqueada, adelantar el reloj 15 min (prueba automatizada) y entrar con datos correctos	Entra al panel y el contador de intentos vuelve a 0	Alta	Pasó

ID	CA	Escenario	Pasos y datos	Resultado esperado	Prioridad	Estado
CP-01-08	CA3	Límite: 4 fallos y luego acierto	Fallar 4 veces y entrar con la contraseña correcta en el 5.º	Entra; la cuenta no se bloquea y el contador vuelve a 0	Media	Pasó
CP-01-09	CA4	Solicitar restablecimiento	Pulsar «¿Olvidaste tu contraseña?» y capturar el correo del productor	Mensaje neutro «Si el correo existe, te enviamos un enlace»; se genera el enlace (en el MVP se muestra en la consola del servidor)	Alta	Pasó
CP-01-10	CA4	Restablecer con enlace válido	Abrir el enlace, capturar `Nueva#2026` y luego iniciar sesión con ella	Entra con la nueva contraseña; la anterior ya no funciona	Alta	Pasó
CP-01-11	CA4	Enlace usado o vencido	Abrir de nuevo el enlace ya usado, o uno con más de 1 h	Mensaje «El enlace ya no es válido»; la contraseña no cambia	Media	Pasó
CP-01-12	CA1	Ruta protegida sin sesión	Abrir directamente `/panel` sin haber iniciado sesión	Redirige a la pantalla de inicio de sesión	Alta	Pasó
CP-01-13	CA1	Contraseña guardada cifrada	Revisar el registro del usuario en la base de datos	El campo guarda un hash bcrypt, nunca la contraseña en texto	Media	Pasó

HU-03 · Registrar parcela

Como productor, quiero registrar mis parcelas con nombre, superficie, cultivo y ubicación, para organizar el monitoreo por terreno.

Criterios de aceptación: **CA1** la parcela capturada completa aparece en el panel · **CA2** con un campo obligatorio faltante no se guarda y se marca el campo · **CA3** los cambios al editar se reflejan de inmediato.

Precondición de todos los casos: sesión iniciada como `productor@agroriego.test`.

ID	CA	Escenario	Pasos y datos	Resultado esperado	Prioridad	Estado
CP-03-01	CA1	Registro completo	Nueva parcela: nombre `El Carrizal`, superficie `6.5` ha, cultivo `Maíz`, ubicación marcada en el mapa; pulsar Guardar parcela	Regresa al panel y El Carrizal aparece con 6.5 ha y Maíz	Alta	Pasó
CP-03-02	CA2	Falta el nombre	Mismos datos sin nombre	No guarda; el campo Nombre se marca en rojo con su mensaje	Alta	Pasó
CP-03-03	CA2	Falta la superficie	Mismos datos sin superficie	No guarda; aparece «Captura la superficie en hectáreas para guardar»	Alta	Pasó
CP-03-04	CA2	Falta la ubicación	Mismos datos sin marcar el mapa	No guarda; se pide marcar la ubicación	Alta	Pasó
CP-03-05	CA2	Superficie cero o negativa	Superficie `0` y luego `-3`	No guarda; mensaje «La superficie debe ser mayor a 0»	Media	Pasó
CP-03-06	CA2	Superficie no numérica (API)	Enviar `superficieHa: "seis"` a POST /parcelas	Respuesta 400 con el campo inválido; no se crea el registro	Media	Pasó
CP-03-07	CA1	Límite: superficie mínima	Superficie `0.1` ha con los demás datos válidos	Se guarda y aparece en el panel	Baja	Pasó
CP-03-08	CA2	Coordenadas fuera de rango (API)	Enviar latitud `91` a POST /parcelas	Respuesta 400; no se crea el registro	Baja	Pasó

ID	CA	Escenario	Pasos y datos	Resultado esperado	Prioridad	Estado
CP-03-09	CA3	Editar superficie y cultivo	Abrir El Carrizal, cambiar a `7` ha y `Trigo`, guardar	El panel muestra 7 ha y Trigo sin recargar la app	Alta	Pasó
CP-03-10	CA3	Edición con campo vacío	Editar El Carrizal y borrar el nombre	No guarda y conserva el nombre anterior	Media	Pasó
CP-03-11	CA1	Aislamiento entre productores	Iniciar sesión como `productor2@agroriego.test`, abrir el panel y pedir GET /parcelas/:id de El Mezquite	El panel no muestra parcelas del productor 1 y la API responde 404	Alta	Pasó

HU-04 · Vincular sensor y válvula

Como técnico de campo, quiero vincular un sensor de humedad y una válvula a una parcela mediante su código, para que la parcela empiece a enviar lecturas y se pueda regar.

Criterios de aceptación: **CA1** el equipo queda vinculado al capturar o escanear su código · **CA2** un código ya vinculado a otra parcela se rechaza con aviso · **CA3** con su primera lectura el sensor pasa a «En línea».

Precondición: sesión iniciada como `tecnico@agroriego.test`, salvo donde se indica otra cosa. Las lecturas se envían con el simulador MQTT.

ID	CA	Escenario	Pasos y datos	Resultado esperado	Prioridad	Estado
CP-04-01	CA1	Vincular sensor capturando el código	Código `HS-0161`, parcela El Carrizal, pulsar Vincular equipo	Mensaje «HS-0161 vinculado a El Carrizal»; estado «Esperando lectura»	Alta	Pasó
CP-04-02	CA1	Vincular sensor escaneando el QR	Escanear el QR impreso de `HS-0162` y elegir La Noria	El código se llena solo y el sensor queda vinculado a La Noria	Alta	Pasó

ID	CA	Escenario	Pasos y datos	Resultado esperado	Prioridad	Estado
CP-04-03	CA1	Vincular válvula	Código `VL-0040`, parcela El Carrizal	La válvula aparece en la lista de equipos ligada a El Carrizal	Alta	Pasó
CP-04-04	CA2	Código ya vinculado a otra parcela	Código `HS-0142` (ya en El Mezquite), parcela Los Álamos	Aviso «Este equipo ya está vinculado a El Mezquite»; la API responde 409 y no cambia nada	Alta	Pasó
CP-04-05	CA2	Código inexistente	Código `HS-9999`	Aviso «No encontramos ese equipo»; no se vincula	Media	Pasó
CP-04-06	CA1	Usuario sin rol de técnico	Como productor, enviar POST /dispositivos/vincular con `HS-0161`	La API responde 403 y el equipo no se vincula	Alta	Pasó
CP-04-07	CA3	Primera lectura cambia el estado	Con `HS-0161` vinculado, publicar desde el simulador humedad 41 %, temperatura 29 °C, batería 95 %	En menos de 10 s el estado cambia a «En línea» y se guarda la lectura	Alta	Pasó
CP-04-08	CA3	Sin lecturas no cambia el estado	Vincular `HS-0162` y no enviar lecturas	Se mantiene «Esperando lectura»	Media	Pasó
CP-04-09	CA3	Lectura de un equipo no vinculado	Publicar una lectura de un código sin vincular	La lectura se descarta y se registra en el log; ninguna parcela cambia	Baja	Pasó

Ejecución de las pruebas

Los 33 casos pasaron el 1 de noviembre de 2026 en GitHub Actions, sobre la rama `sprint-1`: 35 pruebas automatizadas de la API (con PostgreSQL 16 real y un broker MQTT en memoria) y 22 de la interfaz.

Ejecución	Resultado	Evidencia
Corrida 1 (commit 5b91891)	API 35/35 · interfaz 21/22	Actions #41402139029
Corrida 2 (commit 212aa7f)	API 35/35 · interfaz 22/22 · simulador 3/3 · imágenes Docker OK	Actions #41402269550

Defecto encontrado en la corrida 1: la prueba de interfaz de CP-03-02 buscaba el campo de superficie antes de que cargara el formulario. Era un error de la prueba, no de la app; se corrigió en el commit 212aa7f.

Limitaciones: CP-04-02 se probó con un escáner simulado y con la normalización del código en la API; falta escanear un QR físico con la cámara de un celular. Los casos funcionales manuales sobre el ambiente de AWS se repiten cuando esté desplegado (T1.2).

Matriz de trazabilidad

Los 10 criterios de aceptación del Sprint 1 tienen casos que los prueban; ninguno queda sin cubrir.

Historia	Criterio	Casos de prueba	Total
HU-01	CA1 Acceso con datos correctos	CP-01-01, CP-01-02, CP-01-12, CP-01-13	4
HU-01	CA2 Mensaje genérico	CP-01-03, CP-01-04, CP-01-05	3
HU-01	CA3 Bloqueo tras 5 intentos	CP-01-06, CP-01-07, CP-01-08	3
HU-01	CA4 Restablecer contraseña	CP-01-09, CP-01-10, CP-01-11	3
HU-03	CA1 Parcela aparece en el panel	CP-03-01, CP-03-07, CP-03-11	3
HU-03	CA2 Campo obligatorio faltante	CP-03-02, CP-03-03, CP-03-04, CP-03-05, CP-03-06, CP-03-08	6
HU-03	CA3 Edición se refleja	CP-03-09, CP-03-10	2

Historia	Criterio	Casos de prueba	Total
HU-04	CA1 Vincular por código o QR	CP-04-01, CP-04-02, CP-04-03, CP-04-06	4
HU-04	CA2 Código ya vinculado	CP-04-04, CP-04-05	2
HU-04	CA3 Primera lectura → En línea	CP-04-07, CP-04-08, CP-04-09	3
Total			**33**

Cada caso de prioridad Alta también se automatiza en el repositorio (`api/` y `web/`) y su resultado se anota en `docs/pruebas/HU-XX.md`.

Anexo C

Tablero Kanban, Niko-Niko y Burndown chart del Sprint 1

Estado al 8 de noviembre de 2026

AgroRiego — Tablero Kanban

Oct 29, 2026 · @Julian

Métodos Ágiles de Desarrollo · Dr. Carlos Jesús Hinojosa Rodríguez · Equipo 3

Cómo se usa el tablero

Cada historia del sprint se divide en tareas técnicas; cada tarea avanza por 4 columnas y el equipo actualiza su estado en la junta diaria. Todas las tareas empiezan en «Por hacer» y se cambian con el menú de la columna Estado. Este documento es el tablero oficial del equipo; su PDF se publica en el repositorio de GitHub como evidencia para el informe de cada sprint.

Columna	Significado	Límite WIP
Por hacer	Tarea planeada en el sprint, sin empezar	Sin límite
En progreso	Alguien la está trabajando	1 por persona (máx. 5)
En pruebas	Código terminado; se ejecutan sus casos de prueba	3
Hecho	Cumple la Definición de Terminado (DoD)	Sin límite

Reglas de movimiento:

- Una tarea pasa a «En progreso» solo si su responsable no tiene otra tarea en esa columna.
- Pasa a «En pruebas» cuando su código está en un pull request de GitHub.
- Pasa a «Hecho» cuando sus pruebas pasan y el PR se integró a la rama principal. Los defectos encontrados se registran en GitHub Issues con la etiqueta *Bug* ligado a la historia.
- Una historia está terminada cuando todas sus tareas están en «Hecho» y el Product Owner la acepta.

Responsables según especialidad: Alejandro Erro (base de datos), Christian Daniel Valencia (backend y API), Julian Espinosa (nube e IoT), Paul Seañez (frontend y pruebas), Ricardo Orduño (Product Owner, acepta historias).

Sprint 0 · Planeación y diseño (hasta 29 oct 2026)

Objetivo: dejar listo el producto para empezar a construir. Todas las tareas de este sprint están terminadas y sus entregables están en el documento de los puntos 1 a 4.

ID	Tarea	Entregable	Responsable	Estado
T0.1	Redactar historias de usuario con criterios de aceptación	16 historias en 6 épicas	Ricardo Orduño	Hecho
T0.2	Priorizar historias con MoSCoW y valor de negocio	Tabla de priorización	Ricardo Orduño	Hecho
T0.3	Estimar historias con Planning Poker	80 SP estimados	Alejandro Erro, Daniel Valencia, Julian Espinosa, Paul Seañez	Hecho
T0.4	Ordenar el Product Backlog y repartir 3 historias por sprint; definir DoR y DoD	Product Backlog y plan de 3 sprints	Julian Espinosa	Hecho
T0.5	Prototipo de alta fidelidad de la app móvil	8 pantallas navegables	Paul Seañez, Daniel Valencia	Hecho
T0.6	Preparar tableros de seguimiento	Kanban, Niko-Niko y Burndown	Julian Espinosa	Hecho

Sprint 1 · 26 oct – 6 nov 2026

Objetivo: el productor entra, registra sus parcelas y vincula su sensor y válvula. Historias: HU-01, HU-03 y HU-04 (11 SP), más 2 tareas de preparación del proyecto.

ID	Tarea	HU	Responsable	Estado
T1.1	Configurar repositorio en GitHub (github.com/Junsa64/AgroRiego) y tableros de seguimiento	Preparación	Julian Espinosa	Hecho

ID	Tarea	HU	Responsable	Estado
T1.2	Crear ambiente de pruebas en AWS (EC2 con Docker Compose)	Preparación	Julian Espinosa	En progreso
T1.3	Tabla de usuarios con contraseña cifrada	HU-01	Alejandro Erro	En pruebas
T1.4	API de inicio de sesión, bloqueo tras 5 intentos y recuperación por correo	HU-01	Daniel Valencia	En pruebas
T1.5	Pantalla de inicio de sesión	HU-01	Paul Seañez	En pruebas
T1.6	Casos de prueba de HU-01	HU-01	Alejandro Erro	En pruebas
T1.7	Tabla de parcelas	HU-03	Alejandro Erro	En pruebas
T1.8	API para registrar y editar parcelas	HU-03	Daniel Valencia	En pruebas
T1.9	Pantalla de registro de parcela con mapa y validaciones	HU-03	Paul Seañez	En pruebas
T1.10	Casos de prueba de HU-03	HU-03	Alejandro Erro	En pruebas
T1.11	Registro de dispositivos en el servicio IoT de la nube	HU-04	Julian Espinosa	En pruebas
T1.12	API de vinculación de equipo por código o QR	HU-04	Daniel Valencia	En pruebas
T1.13	Pantalla del técnico para vincular equipo	HU-04	Paul Seañez	En pruebas
T1.14	Pruebas con sensor y válvula simulados	HU-04	Julian Espinosa	En pruebas

Estado al 1 de noviembre: el código de T1.3 a T1.14 está en la rama `sprint-1` de GitHub, con commits identificados por tarea, y sus 57 pruebas automatizadas pasan en GitHub Actions. Esas tareas pasan a «Hecho» cuando otro integrante revise y acepte el pull

request a `main`. T1.2 tiene listos Docker, CI y el script de despliegue; queda «En progreso» hasta que la app corra en la instancia EC2 de AWS.

Sprint 2 · 9 – 20 nov 2026

Objetivo: el productor ve la humedad en tiempo real, riega a distancia y recibe alertas. Historias: HU-05, HU-07 y HU-11 (21 SP).

ID	Tarea	HU	Responsable	Estado
T2.1	Recepción de lecturas cada 5 min por MQTT en la nube	HU-05	Julian Espinosa	Por hacer
T2.2	Tabla de lecturas con índices por parcela y fecha	HU-05	Alejandro Erro	Por hacer
T2.3	API de lecturas con actualización en vivo	HU-05	Daniel Valencia	Por hacer
T2.4	Pantalla del panel de parcelas con color por estado	HU-05	Paul Seañez	Por hacer
T2.5	Casos de prueba de HU-05	HU-05	Alejandro Erro	Por hacer
T2.6	Comando para abrir y cerrar válvula con confirmación de estado	HU-07	Julian Espinosa	Por hacer
T2.7	API de riego manual y registro de duración	HU-07	Daniel Valencia	Por hacer
T2.8	Botón de riego con confirmación y aviso de equipo sin conexión	HU-07	Paul Seañez	Por hacer
T2.9	Pruebas con válvula simulada	HU-07	Julian Espinosa	Por hacer
T2.10	Regla de humedad crítica sin repetir la alerta en 1 h	HU-11	Daniel Valencia	Por hacer
T2.11	Notificaciones push	HU-11	Julian Espinosa	Por hacer
T2.12	Bandeja de alertas	HU-11	Paul Seañez	Por hacer

ID	Tarea	HU	Responsable	Estado
T2.13	Casos de prueba de HU-11	HU-11	Alejandro Erro	Por hacer

Sprint 3 · 23 nov – 4 dic 2026

Objetivo: el sistema riega por horario y automáticamente, y avisa fallas de equipo.
Historias: HU-08, HU-09 y HU-12 (16 SP), más la preparación de la demostración final.

ID	Tarea	HU	Responsable	Estado
T3.1	Tabla de programas de riego y validación de empalmes	HU-08	Alejandro Erro	Por hacer
T3.2	Programador de tareas en la nube que abre y cierra la válvula	HU-08	Julian Espinosa	Por hacer
T3.3	Pantalla de agenda y nuevo programa	HU-08	Paul Seañez	Por hacer
T3.4	Casos de prueba de HU-08	HU-08	Alejandro Erro	Por hacer
T3.5	Lógica de riego automático por umbral y nivel objetivo	HU-09	Daniel Valencia	Por hacer
T3.6	Conexión del riego automático con la válvula	HU-09	Julian Espinosa	Por hacer
T3.7	Interruptor de modo automático y configuración de umbrales	HU-09	Paul Seañez	Por hacer
T3.8	Casos de prueba de HU-09	HU-09	Alejandro Erro	Por hacer
T3.9	Monitor de equipos: 30 min sin lecturas y batería menor a 15 %	HU-12	Julian Espinosa	Por hacer
T3.10	Aviso al técnico cuando un equipo falla	HU-12	Daniel Valencia	Por hacer
T3.11	Casos de prueba de HU-12	HU-12	Paul Seañez	Por hacer
T3.12	Preparar la demostración del producto final	Cierre	Ricardo Orduño	Por hacer

AgroRiego — Niko-Niko

Oct 29, 2026 · @Julian

Métodos Ágiles de Desarrollo · Dr. Carlos Jesús Hinojosa Rodríguez · Equipo 3

Cómo se registra

Al terminar cada día de trabajo, cada integrante marca cómo se sintió con el proyecto en su celda del calendario. El Niko-Niko permite ver si el ánimo del equipo baja antes de que se vuelva un impedimento.

Valor	Significado
Feliz	Buen avance, sin bloqueos
Neutral	Avance lento o dudas por resolver
Triste	Bloqueado, sobrecargado o con un problema

Reglas:

- Cada quien marca solo su propia fila, eligiendo el valor en el menú de la celda.
- Si alguien marca «Triste», se comenta en la siguiente junta diaria como impedimento y se anota en la tabla de notas del sprint.
- En la retrospectiva de cada sprint se revisan los días con más caras neutrales o tristes para buscar la causa.
- Un día sin trabajo (asueto) se deja vacío.

Sprint 1 · 26 oct – 6 nov 2026

Integrante	Lun 26	Mar 27	Mié 28	Jue 29	Vie 30	Lun 2	Mar 3	Mié 4	Jue 5
Alejandro Erro	Feliz	Feliz	Feliz	Feliz	Feliz	Feliz	Feliz	Feliz	Feliz
Daniel Valencia	Neutral	Neutral	Neutral	Neutral	Neutral	Neutral	Neutral	Neutral	Neutral
Julian Espinosa	Feliz	Feliz	Feliz	Feliz	Feliz	Feliz	Feliz	Feliz	Feliz
Paul Seañez	Feliz	Feliz	Feliz	Feliz	Feliz	Feliz	Feliz	Feliz	Feliz
Ricardo Orduño	Neutral	Neutral	Neutral	Neutral	Neutral	Neutral	Neutral	Neutral	Neutral

Notas del sprint:

Fecha	Integrante	Motivo	Acción acordada
26 – 30 oct	Daniel Valencia	Neutral toda la semana por la carga de otras materias	Revisar en la junta del lunes 2 si sus 3 tareas de API caben en la semana 2 o se reparte alguna

Sprint 2 · 9 – 20 nov 2026

El lunes 16 de noviembre es asueto (Revolución Mexicana), por eso esa columna queda sin registro.

Integrante	Lun 9	Mar 10	Mié 11	Jue 12	Vie 13	Lun 16	Mar 17	Mié 18	Jue 19	Vie 20
Alejandro Erro	▾	▾	▾	▾	▾	Asueto	▾	▾	▾	▾
Daniel Valencia	▾	▾	▾	▾	▾	Asueto	▾	▾	▾	▾
Julian Espinosa	▾	▾	▾	▾	▾	Asueto	▾	▾	▾	▾
Paul Seañez	▾	▾	▾	▾	▾	Asueto	▾	▾	▾	▾
Ricardo Orduño	▾	▾	▾	▾	▾	Asueto	▾	▾	▾	▾

Notas del sprint:

Fecha	Integrante	Motivo	Acción acordada

Sprint 3 · 23 nov – 4 dic 2026

Integrante	Lun 23	Mar 24	Mié 25	Jue 26	Vie 27	Lun 30	Mar 1	Mié 2	Jue 3	Vie 4
Alejandro Erro	▾	▾	▾	▾	▾	▾	▾	▾	▾	▾
Daniel Valencia	▾	▾	▾	▾	▾	▾	▾	▾	▾	▾
Julian Espinosa	▾	▾	▾	▾	▾	▾	▾	▾	▾	▾
Paul Seañez	▾	▾	▾	▾	▾	▾	▾	▾	▾	▾
Ricardo Orduño	▾	▾	▾	▾	▾	▾	▾	▾	▾	▾

Notas del sprint:

Fecha	Integrante	Motivo	Acción acordada

AgroRiego — Burndown chart

Oct 29, 2026 · @Julian

Métodos Ágiles de Desarrollo · Dr. Carlos Jesús Hinojosa Rodríguez · Equipo 3

Cómo se registra

Cada sprint debe bajar de su total de puntos de historia a 0 en su último día hábil: Sprint 1 de 11 SP, Sprint 2 de 21 SP y Sprint 3 de 16 SP. El Scrum Master anota al cierre de cada día los SP que faltan.

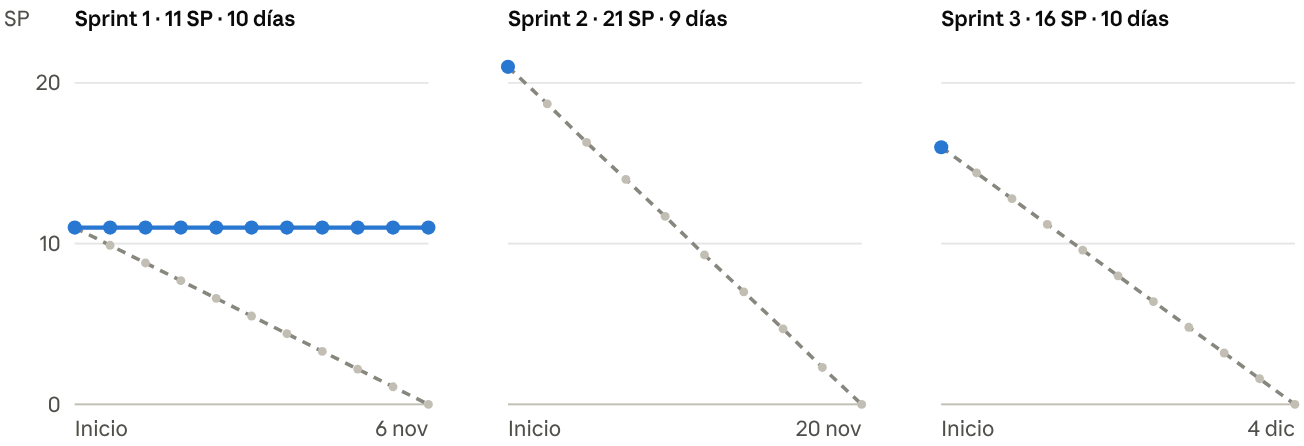
- **SP ideales restantes:** el total del sprint repartido en partes iguales entre sus días hábiles. Ya están calculados en cada tabla.
- **SP reales restantes:** el total del sprint menos los puntos de las historias en «Hecho», es decir, aceptadas por el Product Owner. Una historia a medias no descuenta puntos.
- **Cómo leerlo:** si la línea real queda arriba de la ideal, el equipo va atrasado y se discute en la junta diaria; si queda abajo, va adelantado.

Este documento es el burndown oficial del equipo; su PDF se actualiza en el repositorio de GitHub y sirve como evidencia para el informe de cada sprint.

Gráfica

Cada sprint debe llegar a 0 SP en su último día hábil

SP restantes al cierre de cada día. Punteada: ritmo ideal. Azul: avance real, se agrega con la tabla de cada sprint.



El Sprint 2 tiene 9 días hábiles porque el 16 de noviembre es asueto.

Ideal calculado del Product Backlog (11, 21 y 16 SP) · real: se registra al cierre de cada día

El Sprint 1 cerró el 6 de noviembre con 11 SP sin quemar: el código de las 3 historias estaba listo y probado desde el 1 de noviembre, pero ninguna cumplía la Definición de Terminado porque faltaban la revisión del pull request y el despliegue en AWS. La línea azul se completa con los SP reales de las tablas.

Sprint 1 · 26 oct – 6 nov 2026 · 11 SP

Día	Cierre del día	SP ideales restantes	SP reales restantes	Historias terminadas / notas
0	Inicio del sprint	11.0	11	Planeación del sprint
1	Lun 26 oct	9.9	11	
2	Mar 27 oct	8.8	11	
3	Mié 28 oct	7.7	11	
4	Jue 29 oct	6.6	11	
5	Vie 30 oct	5.5	11	Ninguna historia terminada; el código arranca la semana 2. Riesgo de atraso: se revisa en la junta del lunes 2.
6	Lun 2 nov	4.4	11	Código de las 3 historias en la rama sprint-1 con pruebas en verde (desde el 1 nov); falta PR revisado y despliegue
7	Mar 3 nov	3.3	11	
8	Mié 4 nov	2.2	11	
9	Jue 5 nov	1.1	11	
10	Vie 6 nov	0.0	11	Cierre del sprint: ninguna historia cumple la DoD (PR sin aprobar, sin despliegue en AWS)

Sprint 2 · 9 – 20 nov 2026 · 21 SP

El lunes 16 de noviembre es asueto, así que el ideal se reparte en 9 días hábiles.

Día	Cierre del día	SP ideales restantes	SP reales restantes	Historias terminadas / notas
0	Inicio del sprint	21.0	21	Planeación del sprint
1	Lun 9 nov	18.7		
2	Mar 10 nov	16.3		
3	Mié 11 nov	14.0		
4	Jue 12 nov	11.7		
5	Vie 13 nov	9.3		
6	Mar 17 nov	7.0		
7	Mié 18 nov	4.7		
8	Jue 19 nov	2.3		
9	Vie 20 nov	0.0		

Sprint 3 · 23 nov – 4 dic 2026 · 16 SP

Día	Cierre del día	SP ideales restantes	SP reales restantes	Historias terminadas / notas
0	Inicio del sprint	16.0	16	Planeación del sprint
1	Lun 23 nov	14.4		
2	Mar 24 nov	12.8		
3	Mié 25 nov	11.2		
4	Jue 26 nov	9.6		
5	Vie 27 nov	8.0		

Día	Cierre del día	SP ideales restantes	SP reales restantes	Historias terminadas / notas
6	Lun 30 nov	6.4		
7	Mar 1 dic	4.8		
8	Mié 2 dic	3.2		
9	Jue 3 dic	1.6		
10	Vie 4 dic	0.0		